# Gap Analysis

Outreach and Research Workbench

What works today, what is missing, and why

November 2026

# Contents

1. The short version
2. How to read this document
3. Part A: Technical gaps in the current app
4. Part B: Feature gaps and new ideas
5. Later options and things we should avoid
6. Suggested build order

# 1. The short version

The app takes a spreadsheet of contacts, researches each company and person, writes a personal email for each one, and sends it from the job seeker's own Gmail account. The first half of that works well today. The second half, everything that happens after an email is sent, is mostly missing.

## How the app works today

- Upload a spreadsheet of contacts. Recruiters, hiring managers and referral contacts are all welcome in the plan.
- The app cleans the rows and researches each company and contact.
- It decides who is ready to email and writes a personal email for each one.
- The user connects one or more Gmail accounts with Google sign-in and sends the emails.
- Results appear on the campaign pages.

## Where things stand

| | |
|---|---|
| **Working today** | Spreadsheet upload and cleaning, company and contact research, personalized email writing, connecting several Gmail accounts through Google sign-in, and sending emails right away. |
| **Missing** | A daily sending limit, reading replies, follow-ups, AI reply handling, LinkedIn profile import, a full profile, referral contact support, the opportunity tracker, interview briefs, job matching and time-based insights. |

Three things matter most right now. The app needs a safe daily sending limit, it needs to read replies so the user knows where a human is needed, and it needs follow-ups that stop as soon as someone replies. Everything else builds on those three.

# 2. How to read this document

Part A covers technical gaps in the code, including the Python research and email services, the Node back end and the React front end. Part B covers features the user will see. Every item has the same five parts: what is happening, why, what we want, how to build it, and what it improves.

## Priority

- **High** means needed before the app is used for a real campaign.
- **Medium** means important, but the app can still be used without it.
- **Later** means worth doing, but not until the points above are done.

## What it improves

- **Quality**: better emails, safer accounts, more trust, or more useful results.
- **Speed**: less time spent by the user or by the app.
- **Cost**: lower spending on AI or other services.

File names and code references are included in Part A so the developer can go straight to the right place. Part B is written so anyone can follow it.

# 3. Part A: Technical gaps in the current app

## Summary of all items

| # | What | Type | Priority | Improves |
|---|---|---|---|---|
| A1 | Daily sending limit per Gmail account | Technical | **High** | Quality |
| A2 | Gmail sign-in asks for too much access | Technical | **High** | Quality |
| A3 | Replies are not read at all | Technical | **High** | Quality |
| A4 | AI keys and models only set on the server | Technical | **High** | Cost |
| A5 | Setup documents describe the old Gmail method | Technical | **Medium** | Quality |
| A6 | No data tables for the new features | Technical | **High** | Speed |
| B1 | Sender health monitor and warm-up plan | Feature | **High** | Quality |
| B2 | Full job seeker profile | Feature | **High** | Quality |
| B3 | LinkedIn profile import | Feature | **High** | Speed |
| B4 | Job description matching | Feature | **Medium** | Quality |
| B5 | "Why this line" evidence view | Feature | **Medium** | Quality |
| B6 | Replies inbox with sorting | Feature | **High** | Speed |
| B7 | Automatic follow-ups that stop on reply | Feature | **High** | Speed |
| B8 | One-click AI reply handling | Feature | **High** | Speed |
| B9 | Equal support for all three contact types | Feature | **High** | Quality |
| B10 | Opportunity tracker | Feature | **Medium** | Quality |
| B11 | Interview preparation brief | Feature | **Medium** | Quality |
| B12 | Insights tab by time period | Feature | **Medium** | Quality |

## Detailed items

### A1 No daily sending limit per Gmail account

| | | |
|---|---|---|
| **Type:** Technical | **Priority:** High | **Improves:** Quality (protects accounts) |

| | |
|---|---|
| **What is happening** | Emails are sent by the send service in the Node back end (`backend/services/gmailSendService.js`) with automatic retries. Nothing counts how many emails each Gmail account has sent that day, and nothing stops a campaign before Gmail's own limit is reached. |
| **Why** | The sending path was built to get emails out first. A daily counter was never added, so the app cannot tell how close an account is to its limit. |
| **What we want** | • Count sends for each Gmail account, per day.<br>• Stop at 90% of the daily limit and show a clear message explaining why.<br>• Show a simple progress bar for each account.<br>• Gmail's limits are roughly 500 emails a day for regular accounts and 2,000 for Google Workspace accounts. Confirm current figures with Google before setting the 90% line. |
| **How to build it** | • Create a sends log table in Supabase: one row per email with the account, time, status and Gmail message id.<br>• Before each send in `backend/controllers/pipelineController.js`, count today's rows for that account and skip with a clear reason if the 90% line is reached.<br>• Use one fixed time zone for the day boundary and keep it in settings. |
| **What it improves** | Protects the job seeker's Gmail account from being restricted, and gives an honest picture of how much room is left. |

### A2 Gmail sign-in asks for more access than needed

| **Type:** Technical | **Priority:** High | **Improves:** Quality (trust) |
|---|---|---|

| | |
|---|---|
| **What is happening** | The Gmail sign-in requests full access to the mailbox (scope `https://mail.google.com/` in `backend/controllers/pipelineController.js`). Sending emails needs far less than that. |
| **Why** | One broad permission was requested to get sending working quickly. Google treats full mailbox access as a restricted permission. |
| **What we want** | • Ask only for what each feature needs: a sending permission now, and a read-only permission when reply tracking starts.<br>• Explain each permission in plain words at the moment it is requested.<br>• Google may require a formal app review before people outside the test list can connect. Start that early, because it can take weeks. |
| **How to build it** | • Replace the scope list in the sign-in setup in the same controller with the sending and read-only scopes.<br>• Update the consent text in `frontend/src/components/gmail/GmailAccountsManager.jsx`.<br>• Existing connected accounts must sign in again. Show a clear reconnect prompt. |
| **What it improves** | Builds trust with users and lowers the risk of Google blocking the app. |

### A3 Replies are not read at all

| **Type:** Technical | **Priority:** High | **Improves:** Quality (replies are the goal) |
|---|---|---|

| | |
|---|---|
| **What is happening** | Nothing in the Node back end or the Python services reads Gmail threads or an inbox. A sent email has no link to any reply that comes back. |
| **Why** | Reply tracking was never built. Sending only pushes emails out, so the app does not store thread or message identifiers. |
| **What we want** | • Link every reply to the email and contact it belongs to.<br>• Show new replies in one place.<br>• Flag replies that need a person. |
| **How to build it** | • When sending, save the Gmail thread and message identifiers. Right now sending goes through `nodemailer`, which does not return them. Switch to the Gmail API send call so the identifiers come back.<br>• Create a replies table in Supabase.<br>• Check for replies on a schedule using the Gmail thread list and thread read calls with the read-only permission. Keep only messages the user did not send.<br>• Start with a Sync button. Automatic checks come with the follow-up work (B7). |
| **What it improves** | Makes follow-ups, reply sorting and AI reply handling possible, and lets the user see replies without leaving the app. |

## A4 AI keys and models can only be set on the server

| **Type:** Technical | **Priority:** High | **Improves:** Cost (cheaper models where they are enough) |
|---|---|---|

| | |
|---|---|
| **What is happening** | Each AI task (company research, email writing, role and country, personality analysis) reads its key and model from server settings. Default model names are written into `core/aiClient.py`. The Node back end has a second, separate AI client in `backend/services/aiClient.js`. |
| **Why** | The app was set up by one operator who controls the server. There is no screen for choosing a key or a model, and two AI clients means two places to change the same thing. |
| **What we want** | • A simple mode: one key and one model for everything.<br>• A custom mode: a key and a model chosen for each task.<br>• Keys are stored encrypted, never shown again in full, and never sent back to the browser. |
| **How to build it** | • Create a settings table with encrypted key fields. Use the same encryption approach as the Gmail token storage in `backend/services/gmailCredentialService.js`.<br>• Add one settings reader used by both the Python and Node clients, with the server environment as the fallback.<br>• Add a settings page with the two modes. |
| **What it improves** | Lets the user choose cheaper or stronger models per task, and keeps keys safe. |

## A5 Setup documentation describes the old Gmail method

| **Type:** Technical | **Priority:** Medium | **Improves:** Quality (maintenance) |
|---|---|---|

| | |
|---|---|
| **What is happening** | The README says emails go out through Gmail SMTP with a Google App Password. DEPLOYMENT.md says App Passwords are entered on the Candidate Profile page. The code now uses a Google sign-in connection (`GmailAccountsManager.jsx` and `gmailCredentialService.js`), and the back end README still lists SMTP defaults. |
| **Why** | The move to Google sign-in was made in code, but the documents were not updated afterwards. |
| **What we want** | • The documents match how the app works today, so anyone picking up the project is not misled. |
| **How to build it** | • Update the Gmail sections of the README, DEPLOYMENT and back end README files.<br>• Check whether the older app password fallback in `gmailCredentialService.js` is still needed. If not, remove it together with the old database column. |
| **What it improves** | Saves time for anyone working on the project, including future you. |

### A6 No data tables yet for the new features

| **Type:** Technical | **Priority:** High | **Improves:** Speed (faster to build everything else) |
|---|---|---|

| | |
|---|---|
| **What is happening** | The database has tables for campaigns, candidate assets, company research, decisions, personality analysis, role and country intelligence, and Gmail accounts. It has nothing for replies, follow-ups, opportunities, interview briefs, job matching or LinkedIn profile data. |
| **Why** | These features were not part of the first version of the app. |
| **What we want** | • Separate, clean records for each new feature, added without changing existing data. |
| **How to build it** | • Add each new table through a migration with row level security, so each user only sees their own rows.<br>• Planned tables: sends log (A1), replies (A3), follow-ups (B7), opportunities (B10), interview briefs (B11), profile details (B2) and LinkedIn imports (B3).<br>• Leave existing tables unchanged. |
| **What it improves** | Makes every new feature quicker to add and keeps the data organized. |

## Deferred for now

Two things are deliberately left for later. Always-on background processing (a service that keeps working even when the app is closed) is needed for fully automatic follow-ups, but the first version can rely on scheduled checks. Running many job seekers at the same time is also deferred, since the app is built for one user at a time today.

# 4. Part B: Feature gaps and new ideas

## B1 Sender health monitor and warm-up plan

| **Type:** Feature | **Priority:** High | **Improves:** Quality (account safety) |
|---|---|---|

| | |
|---|---|
| **What is happening** | There is no per-account health view. The user cannot see how each Gmail account is doing before using it. |
| **Why** | A new Gmail account that sends a lot, too fast, can be flagged by Gmail. The app does not guard against this yet. |
| **What we want** | • A health card per account: emails sent today, bounces, failed sends, and a status of healthy, slow down or paused.<br>• A warm-up plan for new accounts, for example starting at 20 a day and adding a few more each day. |
| **How to build it** | • Build on the sends log (A1). Show the card in the Gmail accounts section, and store the warm-up schedule per account. |
| **What it improves** | Keeps accounts safe and shows the user which account to use. |

## B2 Full job seeker profile

| **Type:** Feature | **Priority:** High | **Improves:** Quality (more relevant emails) |
|---|---|---|

| | |
|---|---|
| **What is happening** | The candidate form collects basic details, an email address, a LinkedIn link and uploaded files (`CandidateForm.jsx`). |
| **Why** | An email is only as good as the profile behind it. Salary expectations, work history and preferences are not captured. |
| **What we want** | • A profile with work history, target roles, salary expectations, locations, work preferences and sample job descriptions. |
| **How to build it** | • Add the new fields in short sections, one purpose per screen.<br>• Store them in the profile table (A6) and feed them into the existing positioning and proof point services. |
| **What it improves** | Better matches and more relevant emails. |

### B3 LinkedIn profile import

| **Type:** Feature | **Priority:** High | **Improves:** Speed (less typing) |
|---|---|---|

| | |
|---|---|
| **What is happening** | The app accepts a LinkedIn profile link and checks that it looks right. It does not read anything from the profile. |
| **Why** | LinkedIn does not offer open access to profile data for apps like this, so the import has to use a file the user downloads from their own account. |
| **What we want** | • The user uploads their LinkedIn data archive or a saved PDF of the profile.<br>• The app fills in the profile fields and shows a preview to confirm before saving.<br>• The app reminds the user to refresh the import every few months. |
| **How to build it** | • Add an upload step to the candidate form.<br>• Parse the file on the Python side, next to the existing resume handling in `services/candidate_assets/`.<br>• Keep the preview and confirm step before anything is saved. |
| **What it improves** | Saves typing and keeps the profile up to date. |

### B4 Job description matching

| **Type:** Feature | **Priority:** Medium | **Improves:** Quality (better targeting) |
|---|---|---|

| | |
|---|---|
| **What is happening** | No job description is compared with the candidate profile. |
| **Why** | Matching the role is what makes an email feel written for one specific opening. |
| **What we want** | • The user pastes or uploads a job description.<br>• The app shows how well the profile fits, in plain words (strong, partial or weak), with matching skills and gaps listed. |
| **How to build it** | • Add a new AI task with its own prompt in `prompts/` and a schema in `schemas/`.<br>• Use words instead of a number, because the decision engine already blocks score and rating fields.<br>• Save the result against the contact. |
| **What it improves** | Helps the user target the right openings and see gaps early. |

### B5 "Why this line" evidence view

| **Type:** Feature | **Priority:** Medium | **Improves:** Quality (trust and accuracy) |
|---|---|---|

| | |
|---|---|
| **What is happening** | Generated emails come with some personalization details (`PersonalizationMetadata.jsx`), but as far as we could see the user cannot tell which fact each sentence came from. |
| **Why** | Without sources, the user cannot quickly check whether a claim in the email is correct. |
| **What we want** | • Click any sentence in an email and see where it came from: a company news item, a LinkedIn detail or a profile fact. |
| **How to build it** | • Have the email generator save the source for each sentence.<br>• Show those sources in the personalization panel. |
| **What it improves** | Builds trust and lets the user catch wrong claims before anything is sent. |

### B6 Replies inbox with sorting

| **Type:** Feature | **Priority:** High | **Improves:** Speed (less checking by hand) |
|---|---|---|

| | |
|---|---|
| **What is happening** | None yet. The user has to check Gmail by hand to see who replied (depends on A3). |
| **Why** | Without a sorted inbox, the user spends time on routine replies and can miss the important ones. |
| **What we want** | • One inbox page with replies sorted into: interested, wants more information, not now, out of office, bounced, and needs a person.<br>• Each reply shows the original email beside it, with a reply box. |
| **How to build it** | • Use the replies table (A3).<br>• Classify replies with the AI client using a fixed list of labels. Anything unclear goes to needs a person.<br>• Put this on its own page, so each screen does one job. |
| **What it improves** | The user's time goes only to replies that need it. |

## B7 Automatic follow-ups that stop on reply

| **Type:** Feature | **Priority:** High | **Improves:** Speed (more replies, less manual work) |
|---|---|---|

| | |
|---|---|
| **What is happening** | None yet. Each email goes out once, with no follow-up sequence. |
| **Why** | Many replies come after a second or third message. A follow-up must stop the moment the person replies, so nobody is chased twice. |
| **What we want** | • Set follow-up days per campaign, for example day 4 and day 9.<br>• Follow-ups are skipped automatically when a reply arrives or an email bounces.<br>• The user sees the full schedule before anything is sent. |
| **How to build it** | • Store follow-ups in their own table (A6) with a due time.<br>• Run a scheduled check (for example hourly) that looks for replies first, then sends what is due.<br>• The app has no always-on background service today. The first step can be a check each time the app opens, and the full version needs a scheduled job on the hosting service.<br>• Respect the daily limit from A1. |
| **What it improves** | More replies, with no risk of chasing someone who has already answered. |

## B8 One-click AI reply handling

| **Type:** Feature | **Priority:** High | **Improves:** Speed (saves time, keeps control) |
|---|---|---|

| | |
|---|---|
| **What is happening** | None yet. |
| **Why** | The user wants routine replies handled by the AI, but needs to see what it does and be able to stop it at any time. |
| **What we want** | • A clear switch per campaign, labelled 'AI handles replies', off by default.<br>• When on, the AI answers routine replies in the approved email style.<br>• The AI always stops and asks the user about salary, offers, interview dates and personal questions.<br>• Every AI reply is logged and visible, with a stop button. |
| **How to build it** | • Reuse the email generator with a dedicated reply prompt.<br>• Check every draft against a hard list of stop topics before it is sent.<br>• Log each action, and send through the same daily limit as everything else. |
| **What it improves** | Saves the user time while keeping them in control of what goes out. |

### B9 Equal support for recruiters, hiring managers and referral contacts

| **Type:** Feature | **Priority:** High | **Improves:** Quality (more openings covered) |
|---|---|---|

| | |
|---|---|
| **What is happening** | The app has email templates for recruiters and hiring managers (`templates/recruiter_templates/` and `templates/hiring_manager_templates/`). There is no referral template, and the spreadsheet upload has no way to mark a contact's type. |
| **Why** | The app was first shaped around recruiters, so the other two contact types were added only in part. |
| **What we want** | • Each contact has a type: recruiter, hiring manager or referral.<br>• Each type has its own template, tone and call to action, and all three are reported with equal weight. |
| **How to build it** | • Add a contact type column to the upload column mapping (`services/normalization/column_alias_normalizer.py`).<br>• Add a referral template and type-specific rules in the config files.<br>• Show the type in the results and in the Insights tab (B12). |
| **What it improves** | Covers more openings, and each message fits the person reading it. |

### B10 Opportunity tracker

| **Type:** Feature | **Priority:** Medium | **Improves:** Quality (one place to see progress) |
|---|---|---|

| | |
|---|---|
| **What is happening** | None yet. The results pages show each contact, but there is no way to track where each opportunity stands. |
| **Why** | The user is working several routes at once and needs one view of where each one is. |
| **What we want** | • A simple list with stages (contacted, replied, interview, offer, closed). Each entry links to the contact, the email and the replies. |
| **How to build it** | • Use the opportunities table (A6).<br>• Start with a list and a stage dropdown. A board view can come later. |
| **What it improves** | The user sees every route in one place and nothing gets forgotten. |

### B11 Interview preparation brief

| **Type:** Feature | **Priority:** Medium | **Improves:** Quality (ready faster) |
|---|---|---|

| | |
|---|---|
| **What is happening** | Company research and personality analysis already exist as results, but they are not packaged for interview use. |
| **Why** | When an interview is booked, the user needs everything in one page, not spread across several results. |
| **What we want** | • One brief per opportunity: company summary, recent news, likely questions, talking points from the profile, and questions to ask the interviewer. |
| **How to build it** | • Add a new AI task that reuses the saved company research and personality analysis. Prompt goes in `prompts/`, schema in `schemas/`.<br>• Save each brief against the opportunity. |
| **What it improves** | Gets the user ready quickly once an interview is booked. |

### B12 Insights tab by time period

| **Type:** Feature | **Priority:** Medium | **Improves:** Quality (shows what works) |
|---|---|---|

| | |
|---|---|
| **What is happening** | We did not find a view that breaks results down by time period. |
| **Why** | Without this, the user cannot tell whether the last week was better or worse than before. |
| **What we want** | • Tabs for today, yesterday, last 7 days and this month.<br>• Figures for emails sent, replies, reply rate, follow-ups sent, and contacts by type. |
| **How to build it** | • Query the sends log (A1) and the replies table (A3), grouped by day in one fixed time zone.<br>• Show the figures with the existing interface components. |
| **What it improves** | Shows what is working, so the user can put effort where it pays off. |

# 5. Later options and things we should avoid

## Later options (not now)

- **Paid lead lists.** A bought spreadsheet of contacts. The upload already reads spreadsheets, so a purchased file can be added with a column-mapping step.
- **Paid contact lookup services.** These need a contract and a check that the data can legally be used for outreach. Keep this on the list for later.

## Not recommended

- **Automatic LinkedIn syncing or scraping.** It goes against LinkedIn's terms and can get the user's account restricted. The user-driven import in B3 gives most of the benefit safely.

# 6. Suggested build order

- **Step 1:** Daily sending limit and health card (A1, B1), and update the setup documents (A5).
- **Step 2:** Narrow the Gmail permissions and ask users to reconnect (A2). Start the Google review now.
- **Step 3:** Data tables for the new features (A6), then reply tracking and the inbox (A3, B6).
- **Step 4:** Follow-ups that stop on reply, and the AI reply switch (B7, B8).
- **Step 5:** Full profile and LinkedIn import (B2, B3), plus contact types for all three groups (B9).
- **Step 6:** AI settings for keys and models (A4).
- **Step 7:** Job matching, evidence view, opportunity tracker, interview briefs and insights (B4, B5, B10, B11, B12).